

Name: Kamran Gul

CMS: 023-25-0161

Section: C

OOP Lab: 12

Task 1

Measurable and Rectangle (Interface for “has an area”)

Scenario:

A toolkit treats many objects as **measurable** (they can report an **area**). A **rectangle** is one concrete kind. Model **Measurable** as an **interface** with one operation **area()**. Implement **Rectangle** as a **class** that **implements** **Measurable**.

Requirements:

1. **Interface `Measurable`:**
 - o Method: `double area();` — no body in the interface.
2. **Concrete class `Rectangle` implements `Measurable`:**
 - o Private attributes: `width (double)`, `height (double)`.
 - o Constructor: `Rectangle(double width, double height)`.
 - o `@Override` `public double area()` to return `width * height`.
 - o Add `double perimeter()` returning `2 * (width + height)` (not part of the interface).
3. **In `Main`:**
 - o Declare a reference of type **Measurable** pointing to a **Rectangle**: e.g. `Measurable m = new Rectangle(4.0, 5.0);`
 - o Print `m.area()` (expect **20.0**).
 - o Comment in your report: you **cannot** call `m.perimeter()` through **Measurable m** unless you cast—explain why.

Solution

```
interface Measurable{
    public double area();
}
class Rectangle implements Measurable{
    private double width;
    private double height;
    Rectangle(double width, double height){
        this.width=width;
        this.height=height;
    }
    @Override
    public double area(){
        return width * height;
    }
}
```

```

    }
    public double perimeter(){
        return 2 * (width + height);
    }
}
class Main{
    public static void main(String[] args){
        Measureable m = new Rectangle(4,5);
        System.out.println("Area of Rectangle: "+m.area());
        // System.out.println("Perimeter of Rectangle: "+ m.perimeter());
        // This will not work because we have stored the Rectangle obj in
interface variable
        // Following code can call perimeter
        Rectangle r = new Rectangle(4,5);
        System.out.print(r.perimeter());
    }
}

```

Output

```

Area of Rectangle: 20.0
18.0

```

Task 2

Payable and HourlyService (Interface for “amount due”)

Scenario:

Accounting software processes anything **payable**—each object must report **how much is due now**. An **hourly service** bill charges **hours × rate**. Model ``Payable`` with ``amountDue()``. Implement ``HourlyService``.

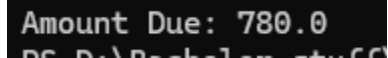
Requirements:

1. **Interface ``Payable``:**
 - Method: `double amountDue();`
2. **Concrete class ``HourlyService`` implements ``Payable``:**
 - Private attributes: `hours` (double), `ratePerHour` (double).
 - Constructor: `HourlyService(double hours, double ratePerHour).`
 - ``@Override`` `public double amountDue()` to return `hours * ratePerHour`.
3. **In ``Main``:**
 - Create `Payable p = new HourlyService(10, 50);`
 - Print `p.amountDue()` (expect **500.0**).

Solution

```
interface Payable{
    public double amountDue();
}
class HourlyService implements Payable{
    private double hours;
    private double ratePerHour;
    HourlyService(double hours, double ratePerHour){
        this.hours = hours;
        this.ratePerHour = ratePerHour;
    }
    @Override
    public double amountDue(){
        return hours * ratePerHour;
    }
}
class Main{
    public static void main(String[] args){
        Payable p = new HourlyService(12,65);
        System.out.print("Amount Due: "+ p.amountDue());
    }
}
```

Output



```
Amount Due: 780.0
```

Task 3

Describable and Product (Interface for “text description”)

Scenario:

A catalog UI can show a **short description** for any **describable** item. A **product** has a **name** and **SKU**. Model ``Describable`` with ``describe()``. Implement ``Product``.

Requirements:

1. **Interface ``Describable``:**
 - o Method: `String describe();`
2. **Concrete class ``Product`` implements ``Describable``:**
 - o Private attributes: `name (String)`, `sku (String)`.
 - o Constructor: `Product(String name, String sku)`.
 - o ``@Override`` `public String describe()` to return something like `"Product: " + name + " (" + sku + ")"` (exact format is flexible but must include name and sku).
3. **In ``Main``:**
 - o Create `Describable d = new Product("Mouse", "SKU-001");`
 - o Print `d.describe()`.

Solution

```
interface Describe{
    public String describe();
}
class Product implements Describe{
    private String name;
    private String sku;
    Product(String name,String sku){
        this.name= name;
        this.sku = sku;
    }
    @Override
    public String describe(){
        return "Product: " + name + " (" +sku+ ")";
    }
}
class Main{
    public static void main(String[] args){
        Describe d = new Product("Mouse","SKU-001");
        System.out.print(d.describe());
    }
}
```

Output

```
Product: Mouse (SKU-001)
```